

Riassunto - Teoria della Complessità

La teoria della complessità classifica i problemi in base alla difficoltà di risolverli efficientemente. Si concentra sui **problemi decisionali** (output SI o NO), poiché la trattazione formale è più agevole e permette di trarre conclusioni applicabili anche ad altri tipi di problemi.

Esempio: Il problema **Shortest Path** può essere espresso sia come problema di ottimizzazione (qual è la lunghezza del cammino minimo?) che come problema decisionale (esiste un cammino di lunghezza $\leq k$?).

Classificazione dei problemi

Classe P (Polynomial Time) - "I Facili"

Sono i problemi che un computer può **RISOLVERE** velocemente (in tempo polinomiale, $O(n^k)$).

- **Esempio:** Ordinare una lista (Sorting), trovare il cammino minimo (Dijkstra).
- **Nota:** Se un problema è in P, è automaticamente anche in NP.

Classe NP (Nondeterministic Polynomial Time) - "I Verificabili"

Sono i problemi per cui, se io ti do una soluzione ("il certificato"), tu puoi **VERIFICARE** velocemente se è corretta con un algoritmo deterministico in tempo polinomiale.

- *Definizioni utili*
 - **Certificato:** sequenza di caratteri di dimensione al massimo polinomiale che contiene l'evidenza che un'istanza i sia positiva per il problema Π .
 - **Algoritmo verificatore:** algoritmo decisionale che prende in input un'istanza i e un certificato C_i , restituendo SI se i è positiva per Π , NO altrimenti.
- **Esempio:** Fare un Sudoku è difficile (ci mette tempo a risolverlo), ma se ti do un Sudoku già compilato, ci mette un attimo a controllare se le regole sono rispettate.
- **Relazione:** $P \subseteq NP$. (Se so risolvere un problema velocemente, so anche verificare la soluzione velocemente).

Classe NP-Hard - "I Più Cattivi di Tutti"

Qui usciamo dal recinto dei problemi "trattabili". Un problema è NP-Hard se è **almeno** difficile quanto il problema più difficile in NP.

- **La Chiave (Riduzione):** Un problema H è NP-Hard se **ogni** problema in NP può essere ridotto (trasformato) in H in tempo polinomiale.
- **Significato:** Se trovassi un modo veloce per risolvere un problema NP-Hard, avrei trovato un modo veloce per risolvere *tutti* i problemi in NP (e quindi avrei dimostrato che $P=NP$).
- **Nota:** Un problema NP-Hard **non deve per forza essere in NP**. Potrebbe essere così difficile che non riesci nemmeno a verificare la soluzione velocemente (o potrebbe essere indecidibile, come l'Halting Problem).

Classe NP-Complete - "L'Intersezione Critica"

Questi sono i problemi "chiave". Sono sia in NP che NP-Hard.

- **Definizione:** Un problema C è NP-Complete se soddisfa due condizioni:
 1. $C \in NP$ (è verificabile efficientemente).
 2. $C \in NP\text{-Hard}$ (tutti i problemi in NP si riducono a lui).
- **Concetto Chiave:** Rappresentano il "nucleo" della difficoltà di NP.
- **Esempi:** Vertex Cover, 3-SAT, TSP Decisionale.

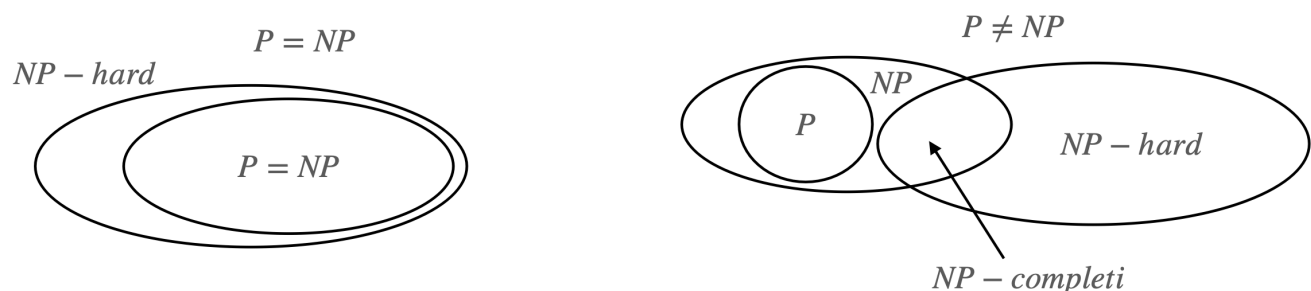
Domanda: Il problema del commesso viaggiatore (TSP) è NP-Complete?

- **TSP Decisionale:** "Esiste un ciclo di costo minore di k ?" → **NP-Complete**. (Posso verificare se un ciclo dato costa meno di k).
- **TSP di Ottimizzazione:** "Trovami il ciclo di costo minimo assoluto." → **NP-Hard**, ma **non** NP-Complete.
 - *Perché?* Se ti do una soluzione e ti dico "Questa è la minima", tu non hai un modo veloce per verificare che non ne esista una ancora più piccola da qualche parte. Devi fidarti. Quindi non è in NP (come definito classicamente per i problemi decisionali).

Come rappresentiamo queste classi di problemi mantenendo il dubbio che in futuro si possano trovare degli algoritmi polinomiali risolutori dei problemi NP?

A sinistra siamo nel caso in cui i problemi NP potrebbero essere risolti in tempo polinomiale, a destra nel caso sia stato dimostrato che non esiste un algoritmo polinomiale per i problemi NP.

SCENARI POSSIBILI



Classe	Trovare Soluzione (Tempo Polinomiale)	Verificare Soluzione (Tempo Polinomiale)	Relazione Insiemistica
P	Sì	Sì	$P \subseteq NP$
NP	NO (o non lo sappiamo)	Sì	Contiene P e NP-Complete
NP-Complete	NO	Sì	Intersezione tra NP e NP-Hard
NP-Hard	NO	NO (non garantito)	Almeno difficile quanto ogni problema in NP

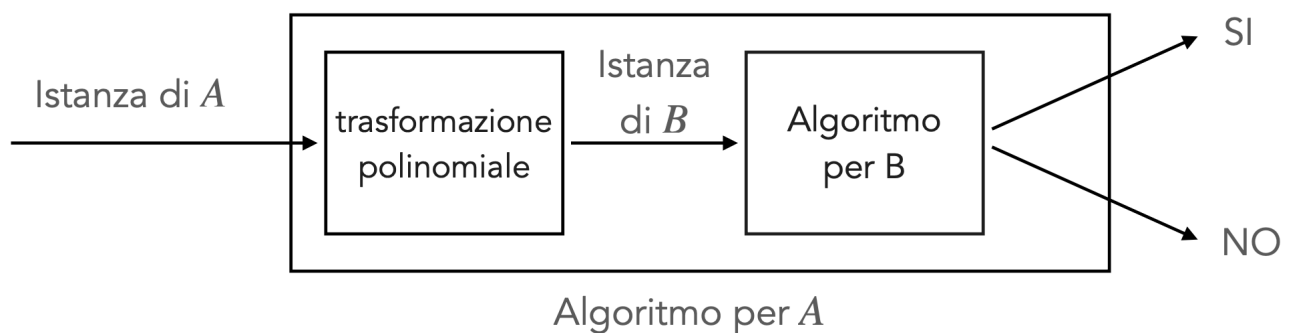
Confrontiamo la difficoltà di due algoritmi: Riduzione di Karp

Definizione: Un problema decisionale A è **riducibile in tempo polinomiale** a B ($A \leq_p B$) se:

- Ogni istanza di A può essere *trasformata* in tempo polinomiale in un'istanza di B
- Ogni istanza positiva di A viene trasformata in un'istanza positiva di B
- Ogni istanza negativa di A viene trasformata in un'istanza negativa di B

Intuizione: $A \leq_p B$ significa che:

- Il problema B non è più facile di A (un algoritmo per B può risolvere A)
- Il problema A non è più difficile di B

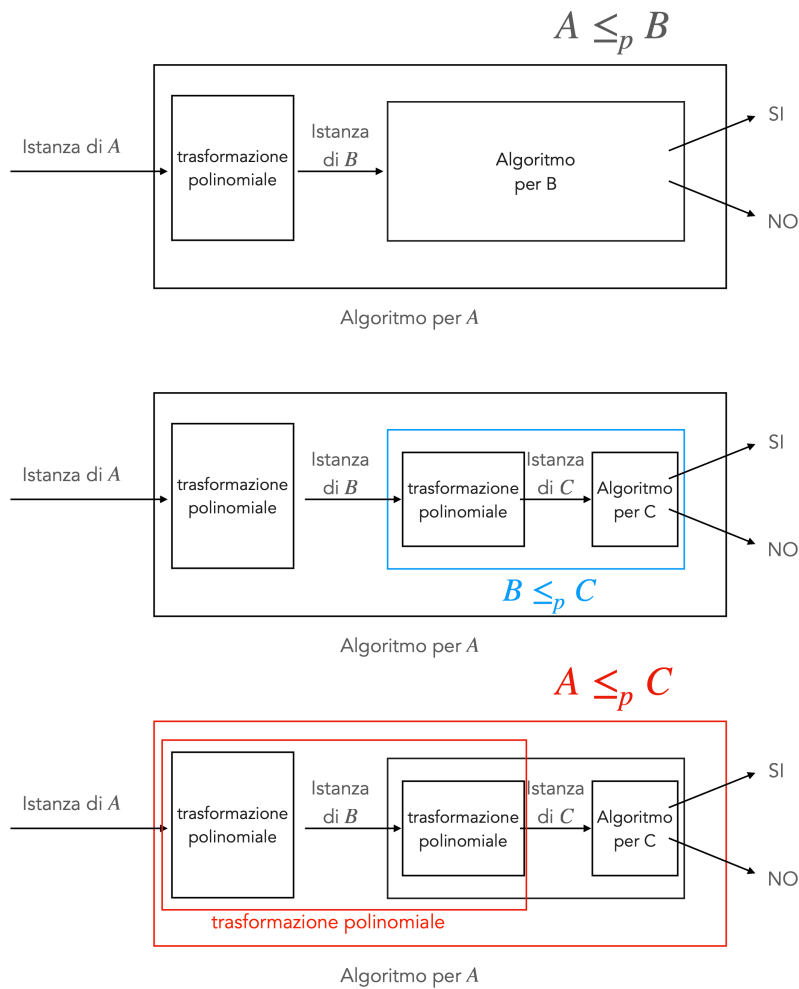


Conseguenze

Se $A \leq_p B$ allora:

- $B \in P \Rightarrow A \in P$ 🚀 abbiamo un algoritmo polinomiale per A 🚀
- $A \notin P \Rightarrow B \notin P$ non abbiamo un algoritmo polinomiale per A 😞, quindi neanche per B 😞
- $B \leq_p A \Rightarrow A \equiv B$ i problemi sono equivalentemente difficili

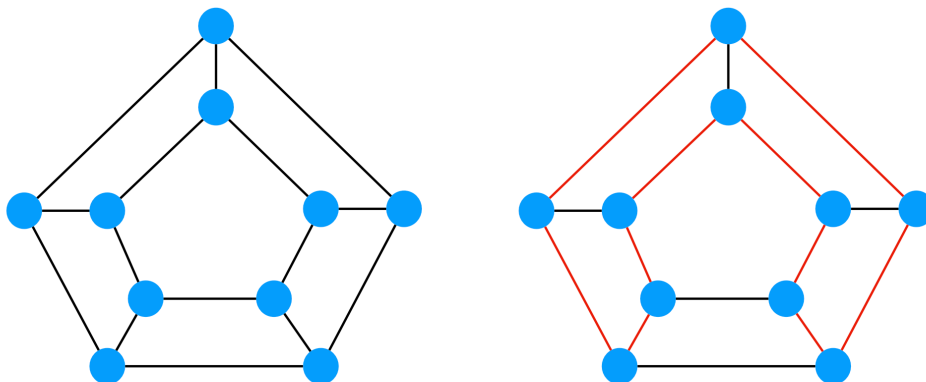
Proprietà: La riduzione polinomiale è **transitiva**: $A \leq_p B \wedge B \leq_p C \Rightarrow A \leq_p C$.



Classe NP (Non-deterministic Polynomial Time)

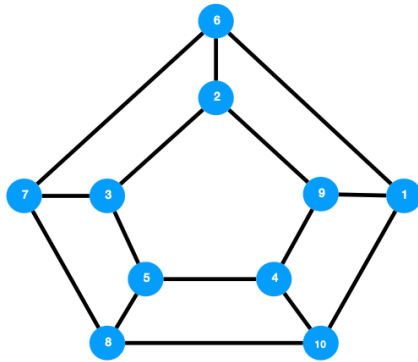
Esempio: Ciclo Hamiltoniano

- **INPUT:** Grafo $G = (V, E)$
- **OUTPUT:** Esiste un ciclo in G che passa per tutti i nodi esattamente una volta?



Per questo problema:

- Non si conoscono algoritmi polinomiali
- Un algoritmo forza bruta richiede tempo **fattoriale**
- Possiamo **verificare** una soluzione in tempo polinomiale: data una permutazione dei nodi, possiamo verificare in tempo lineare se rappresenta un ciclo hamiltoniano



Permutazione: 8,5,3,2,9,4,10,1,6,7

Classe NP-Completo

Definizione: Un problema decisionale A è **NP-completo** se:

1. $A \in NP$
2. $\forall B \in NP : B \leq_p A$

Come dimostrare che un Problema è NP-Completo

Per dimostrare che A è NP-completo, sfruttiamo la transitività della riduzione:

1. Dimostrare che $A \in NP$
2. Scegliere un problema B già dimostrato NP-completo
3. Dimostrare che $B \leq_p A$

Teoremi di Cook (1971) e Levin (1973): Il primo problema dimostrato NP-completo è **SAT** (Satisfiability).

SAT: Data una formula in forma normale congiuntiva (FNC), esiste un assegnamento di verità che la renda vera?

- Esempio: $(a \vee b \vee c) \wedge (\neg b \vee c) \wedge (\neg a \vee \neg b \vee c)$

P.S. and $\rightarrow \wedge$

or $\rightarrow \vee$

Teorema Fondamentale

Se si dimostra che un problema NP-completo:

1. Ha un algoritmo polinomiale, allora $P = NP$
2. Non ha algoritmi polinomiali, allora $P \neq NP$

Classe NP-Hard

Definizione: Un problema A (non necessariamente decisionale) è **NP-hard** se:

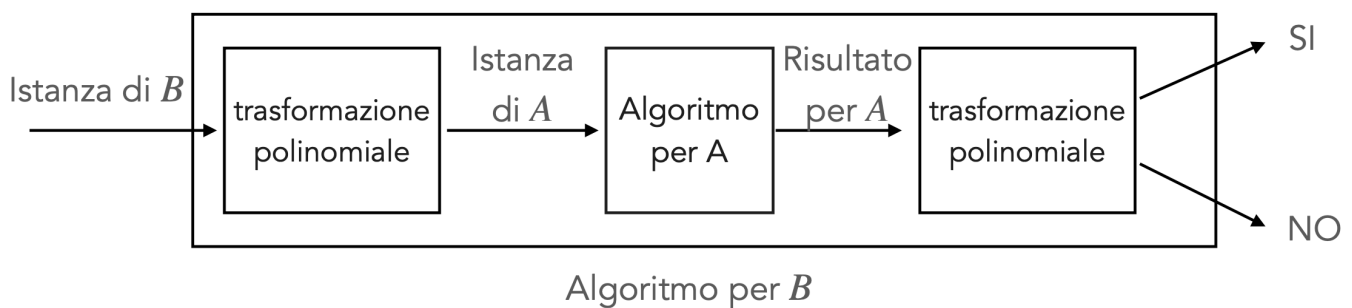
- $\forall B \in NP : B \leq_p A$

Differenza con NP-completo: un problema NP-hard non è necessariamente in NP (ad esempio, problemi di ottimizzazione).

Come Dimostrare che un Problema è NP-Hard

Per dimostrare che A è NP-hard:

1. Scegliere un problema B già dimostrato NP-completo
2. Dimostrare che $B \leq_p A$



N.B.: Non è detto che A abbia un algoritmo verificatore polinomiale

Soluzioni per Problemi NP-Hard

Molti problemi NP-hard hanno importanti applicazioni e non possono essere ignorati. Possibili soluzioni:

- Risolvere **efficientemente solo istanze piccole**
- Utilizzare **euristiche**
- Utilizzare algoritmi **paralleli/distribuiti**

- Utilizzare **algoritmi di approssimazione**: restituiscono in tempo polinomiale una soluzione ammissibile con garanzie sul discostamento dal costo ottimo

Teorema: Il problema del Commesso Viaggiatore (TSP) è NP-Hard

Obiettivo: Dimostrare che $TSP(Optimization) \geq_p HC(decisionale)$.

Vogliamo dimostrare che se avessimo un oracolo (un algoritmo efficiente) in grado di risolvere il TSP, potremmo usarlo per risolvere il problema del Ciclo Hamiltoniano (HC).

Poiché il problema del Ciclo Hamiltoniano (HC) è noto essere NP-Completo, se dimostriamo che esso è riducibile polinomialmente al TSP di ottimizzazione, allora il TSP è almeno tanto difficile quanto un problema NP-Completo, dunque è NP-Hard.

1. Intuizione e Strategia della Riduzione

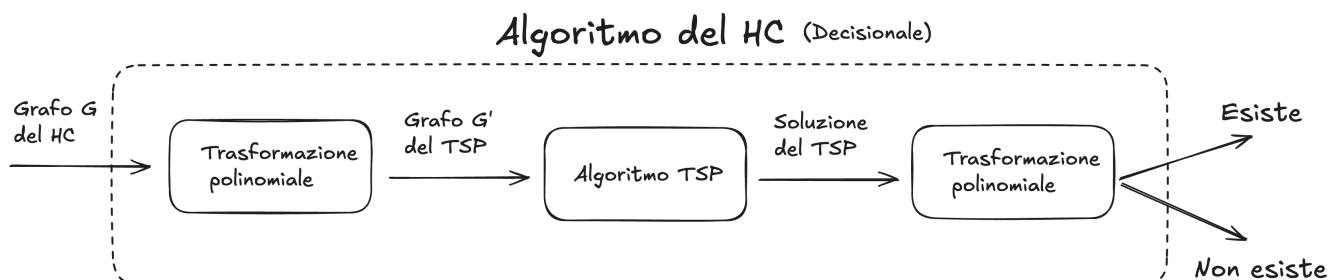
Il problema HC è un problema decisionale (Esiste un ciclo? Sì/No), mentre il TSP è un problema di ottimizzazione (Trova il ciclo di costo minimo).

Per collegarli, dobbiamo trasformare la struttura topologica del grafo di HC in costi per il TSP.

L'idea chiave è penalizzare l'uso di archi non esistenti:

- Assegniamo costo **0** agli archi che esistono realmente (nessuna penalità).
- Assegniamo costo **1** agli archi che non esistono (penalità).

In questo modo, il costo totale del tour fungerà da "rivelatore": se il costo è 0, il tour non ha mai "barato" (ha usato solo archi esistenti); se il costo è ≥ 1 , il tour è stato costretto a "barare" (ha usato archi che non c'erano in origine).



2. Definizione Formale della Riduzione

Costruiamo un algoritmo di trasformazione f che mappa un'istanza G di HC in un'istanza G' di TSP.

- **Input:** Grafo $G = (V, E)$.

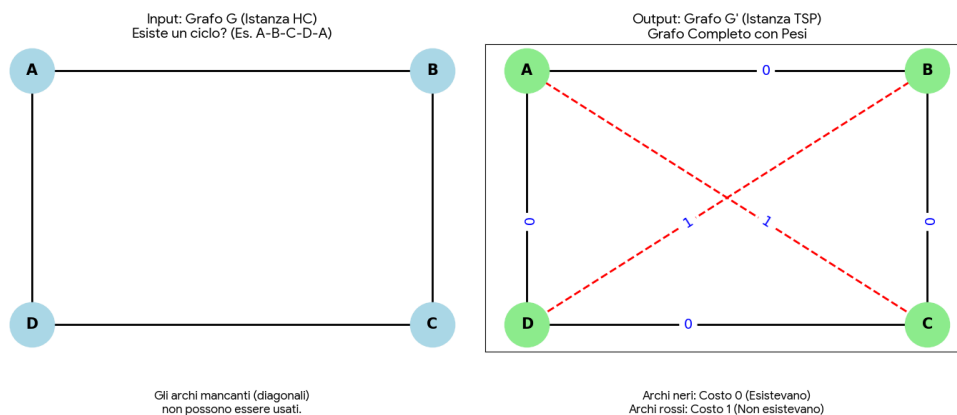
- **Output:** Grafo completo ponderato $G' = (V, E')$ con funzione costo c .

Costruzione di G' :

1. **Vertici:** $V' = V$ (stessi nodi).
2. **Archi:** $E' = V \times V$ (grafo completo: colleghiamo tutto con tutto).
3. **Pesi:** Per ogni coppia (u, v) , il peso è definito come:

$$c(u, v) = \begin{cases} 0 & \text{se } (u, v) \in E \text{ (arco originale)} \\ 1 & \text{se } (u, v) \notin E \text{ (arco fittizio)} \end{cases}$$

_Nota: La costruzione di G' richiede di iterare su tutte le coppie di nodi, impiegando un tempo $O(|V|^2)$, che è polinomiale.



3. Dimostrazione di Correttezza

Sia C_{min} il valore della soluzione ottima restituita dal TSP su G' .

Dobbiamo dimostrare che $C_{min} = 0 \iff G$ ha un HC.

Direzione A

Ipotesi: G ha un HC $\implies$ **Tesi:** $C_{min} = 0$

- **Ragionamento:** Se G ha un ciclo Hamiltoniano, esiste una permutazione di vertici collegati solo da archi in E .
- **Nel grafo TSP:** Poiché abbiamo mappato tutti gli archi di E con peso 0, questo ciclo esiste anche in G' e ha costo somma 0. Dato che i pesi sono non-negativi, 0 è il minimo possibile.
- **Conclusione:** L'algoritmo TSP troverà questo tour (o uno equivalente) e restituirà 0.

Direzione B

Ipotesi: $C_{min} = 0 \implies$ **Tesi:** G ha un HC

(Utilizziamo l'implicazione diretta sul valore del costo, che è più intuitiva della contronominale per chi legge).

- **Ragionamento:** Supponiamo che l'algoritmo TSP restituisca un tour di costo 0.
- **Analisi dei pesi:** Poiché la somma dei pesi è 0 e i pesi possibili sono solo $\{0, 1\}$, questo implica che **ogni singolo arco** del tour ha peso 0.
- **Legame con G :** Per costruzione, un arco ha peso 0 in G' solo se esisteva in G .
- **Conclusione:** Il tour trovato dal TSP è quindi composto esclusivamente da archi originali di G . Poiché un tour TSP visita ogni nodo esattamente una volta, questo corrisponde esattamente alla definizione di Ciclo Hamiltoniano in G .

Nota logica

Dimostrando $C_{min} = 0 \implies G$ ha un HC abbiamo dimostrato anche la sua contronominale:

Ipotesi: G non ha un HC $\implies$ **Tesi:** $C_{min} \neq 0$ (C non può essere negativa, quindi $C_{min} \geq 1$)

Infatti, se $C_{min} \geq 1$, significa che ogni possibile tour deve usare almeno un arco fittizio, quindi non esiste un HC in G .

4. Conclusione

La riduzione è polinomiale e valida. Risolvere il TSP permette di decidere HC:

- Se $TSP(G') = 0 \rightarrow$ Risposta HC: **SÌ**.
- Se $TSP(G') \geq 1 \rightarrow$ Risposta HC: **NO**.

Poiché HC è NP-Completo, **TSP (Ottimizzazione) è NP-Hard**.

Approfondimento: Perché scegliere pesi 1 e 2 invece di 0 e 1?

La scelta dei pesi nella riduzione non è casuale, ma determina **quale variante** del TSP stiamo dimostrando essere NP-Hard.

1. Il Test della Disuguaglianza Triangolare

La disuguaglianza triangolare è la regola che rende un grafo "geometricamente sensato". Essa afferma che andare direttamente da A a B non deve mai costare più che passare attraverso un intermedio C :

$$c(u, w) \leq c(u, v) + c(v, w)$$

Analizziamo i due casi:

Caso A: Pesi $\{0, 1\}$ (La riduzione "Base")

- Arco presente (u, v) e $(v, w) \rightarrow$ costo 0.
- Arco mancante $(u, w) \rightarrow$ costo 1.
- **Verifica:** $1 \leq 0 + 0 \implies 1 \leq 0$ (**FALSO!**)
- **Conclusione:** Questa riduzione genera un grafo che viola la geometria euclidea. Stiamo dimostrando la difficoltà del **TSP Generale**.

Caso B: Pesi $\{1, 2\}$ (La riduzione "Metrica")

- Arco presente (u, v) e $(v, w) \rightarrow$ costo 1.
- Arco mancante $(u, w) \rightarrow$ costo 2.
- **Verifica:** $2 \leq 1 + 1 \implies 2 \leq 2$ (**VERO!**)
- **Conclusione:** Questa riduzione genera un grafo che rispetta la geometria. Stiamo dimostrando la difficoltà del **Metric-TSP**.

2. Perché questa distinzione è cruciale? (Il vero motivo teorico)

La distinzione riguarda l'**approssimabilità**:

1. TSP Generale (Pesi 0/1):

È un problema "cattivo". Non solo è NP-Hard, ma non è nemmeno approssimabile.

Teorema: Se esistesse un algoritmo che approssima il TSP generale entro un fattore polinomiale, allora $P = NP$.

Quindi, con pesi 0/1 dimostriamo che il caso peggiore assoluto è intrattabile.

2. Metric-TSP (Pesi 1/2):

È un problema "più gentile". È NP-Hard (come dimostrato dalla riduzione con pesi 1/2), ma appartiene alla classe APX.

Esistono algoritmi efficienti che garantiscono una soluzione vicina all'ottimo:

- **2-Approximation:** Basato sull'Albero di Copertura Minimo (MST).
- **Algoritmo di Christofides:** Garantisce un'approssimazione di fattore 1.5 (o $3/2$).

Fattore di approssimazione (questa parte devo ancora capirla)

Il **fattore di approssimazione** (spesso indicato con la lettera greca ρ , "rho", o α) è la misura della "garanzia di qualità" che un algoritmo ci offre nel caso peggiore.

Definizione Formale di Algoritmo di Approssimazione

Sia Π un problema di ottimizzazione (minimizzazione o massimizzazione) e sia I una generica istanza di questo problema.

Indichiamo con:

- $OPT(I)$: il valore della soluzione ottima per l'istanza I .
- $A(I)$: il valore della soluzione restituita dal nostro algoritmo (deterministico) A per l'istanza I .

Diciamo che l'algoritmo A è un **algoritmo di ρ -approssimazione** (o ha un fattore di approssimazione ρ) se, per **ogni** istanza I di dimensione n , vale la seguente relazione:

$$A(I) \leq \rho \cdot OPT(I) \Rightarrow \frac{A(I)}{OPT(I)} \leq \rho(n) \quad (\text{per problemi di minimizzazione})$$

oppure

$$A(I) \geq \rho \cdot OPT(I) \Rightarrow \frac{OPT(I)}{A(I)} \leq \rho(n) \quad (\text{per problemi di massimizzazione})$$

In entrambi i casi, il rapporto è definito in modo da essere sempre ≥ 1 .

- Se $\rho(n) = 1$, l'algoritmo è esatto (trova l'ottimo).
- Se $\rho(n) > 1$, l'algoritmo è approssimato. Più ρ è vicino a 1, migliore è l'algoritmo.

Nota Bene:

Il fattore ρ è una garanzia nel caso peggiore (worst-case).

Non stiamo dicendo che l'algoritmo sbaglia sempre di un fattore ρ . Stiamo dicendo che non sbaglierà mai più di un fattore ρ , nemmeno nell'istanza più "cattiva" possibile che l'esaminatore possa inventare.

Tipologie di Fattori di Approssimazione

Nel nostro corso, incontriamo principalmente tre classi di ρ :

1. Fattore Costante ($\rho = k$):

Il caso ideale per un problema NP-Hard. L'errore non cresce con la dimensione dell'input.

- *Esempio:* **Vertex Cover** ha $\rho = 2$.
- *Esempio:* **Metric TSP** (con disuguaglianza triangolare) ha $\rho = 2$ (o $\rho = 1.5$ con Christofides).

2. Fattore Logaritmico ($\rho = O(\log n)$):

L'errore cresce lentamente al crescere dell'input.

- *Esempio classico: Set Cover* (Copertura di Insiemi). Non puoi avere un fattore costante qui (salvo $P=NP$), il meglio che possiamo fare è $\ln n$.

3. Fattore Polinomiale ($\rho = O(n^k)$):

L'approssimazione degrada molto rapidamente. Spesso è inutile in pratica, ma teoricamente interessante.

- *Esempio: General TSP* (senza disuguaglianza triangolare). Qui il fattore di approssimazione può essere arbitrariamente grande (2^n , o peggio), rendendo il problema *inapprossimabile*.

La Tecnica Formale di Dimostrazione (Il "Lower Bound")

Come facciamo a calcolare ρ se non conosciamo $OPT(I)$?

Formalizziamo il ragionamento che abbiamo fatto prima.

Per dimostrare che un algoritmo A è una ρ -approssimazione per un problema di **minimizzazione**, dobbiamo trovare un **Lower Bound (LB)** tale che:

$$LB(I) \leq OPT(I)$$

E dimostrare che il nostro algoritmo soddisfa:

$$A(I) \leq \rho \cdot LB(I)$$

Combinando le due disuguaglianze:

$$\begin{aligned} A(I) &\leq \underbrace{\rho \cdot LB(I)}_{\text{Limite calcolabile}} \leq \underbrace{\rho \cdot OPT(I)}_{\text{Limite ideale}} \\ \Rightarrow \frac{A(I)}{OPT(I)} &\leq \rho \end{aligned}$$

Q.E.D. (Come Volevasi Dimostrare).

Esempio: Il Vertex Cover

Applichiamo il rigore al Vertex Cover.

1. Algoritmo A :

- Trova un *Matching Massimale* M nel grafo $G = (V, E)$.

- Restituisci l'insieme S composto da *tutti* gli estremi degli archi in M .
- Costo: $|S| = 2 \cdot |M|$.

2. Il Lower Bound (LB):

- Sia S^* la soluzione ottima (il Vertex Cover minimo).
- Poiché gli archi in M sono disgiunti (non condividono vertici), per coprirli tutti S^* deve contenere almeno un vertice per ogni arco in M .
- Formalmente: $|S^*| \geq |M|$. Questo è il nostro LB.

3. Calcolo di ρ :

$$\frac{A(I)}{OPT(I)} = \frac{|S|}{|S^*|} \leq \frac{2 \cdot |M|}{|M|} = 2$$

Quindi, l'algoritmo è una **2-approssimazione**.

Vertex cover

Dato un grafo non orientato $G = (V, E)$:

- Un **Vertex Cover** è un sottoinsieme di vertici $V' \subseteq V$ tale che per ogni arco $(u, v) \in E$, si ha che $u \in V'$ oppure $v \in V'$ (o entrambi).
- **Problema di Ottimizzazione:** Trovare il V' con la cardinalità minima (minimo numero di vertici).
- **Problema Decisionale:** Dato G e un intero k , esiste un Vertex Cover di dimensione $\leq k$?

La Complessità (Perché è "difficile"?)

Nel nostro corso classifichiamo il problema decisionale del Vertex Cover come **NP-Completo**.

Cosa significa per il tuo esame?

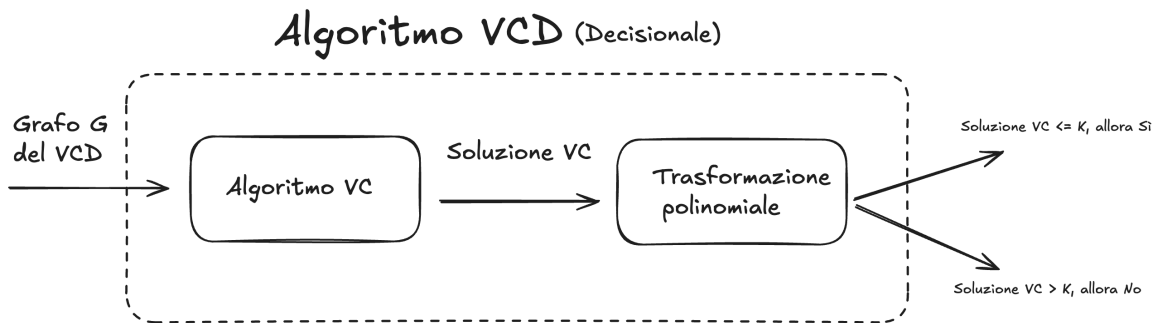
1. È in **NP**: Se ti do una soluzione (un insieme di vertici), puoi verificare in tempo polinomiale se copre tutti gli archi.
2. È **NP-Hard**: È difficile quanto il problema più difficile in NP (spesso si dimostra riducendo il problema della *Clique* o *3-SAT* al Vertex Cover).

In pratica: non conosciamo (e probabilmente non esiste) un algoritmo che trovi la soluzione *esatta* (il minimo assoluto) in tempo polinomiale. Se il grafo è grande, trovare l'ottimo richiederebbe tempi astronomici.

Teorema: Il problema del Vertex Cover (ottimizzazione) è NP-Hard.

Dato che la versione decisionale del Vertex Cover è NP-completo, definiamo la riduzione $VCD \leq_p VC$.

Il Vertex Cover (ottimizzazione) è difficile almeno quanto il Vertex Cover Decisionale.



La riduzione è molto semplice perché l'istanza dell'input dei due algoritmi è la stessa, ci serve solo definire una trasformazione polinomiale dell'output.

Quindi:

1. Utilizziamo l'algoritmo VC sul grado G per trovare il Vertex Cover $V' \subseteq V$ di cardinalità minima
2. Definiamo la trasformazione polinomiale dell'output:
 - se $|V'| \leq k$ allora rispondo **si**.
 - se $|V'| > k$ allora rispondo **no**.

Una volta trovato il Vertex Cover con cardinalità minima, possiamo confrontarlo con k e quindi fornire la risposta decisionale del Vertex Cover.

Il numero di chiamate dell'algoritmo dell'algoritmo è costante (una volta sola viene chiamato) e anche il numero di operazioni extra è costante.

Dunque possiamo definire anche l'algoritmo del Vertex Cover np-hard.

Approfondimento

Possiamo definire anche dimostrare il contrario: $VC \leq_p VCD$

Anche la versione decisionale del Vertex Cover è difficile almeno quanto la versione di ottimizzazione.

Self-Reduction di Vertex Cover (Approfondimento)

Dimostriamo che il problema del Vertex Cover di ottimizzazione è egualmente difficile come il Vertex Cover decisionale.

Il problema di ricerca del Vertex Cover (VC_{search}) è polinomialmente riducibile al problema decisionale del Vertex Cover (VC_{decision}).

$$VC_{\text{search}} \leq_P VC_{\text{decision}}$$

Ciò implica che se esiste un algoritmo polinomiale per decidere se esiste una copertura di dimensione k , esiste anche un algoritmo polinomiale per trovare i vertici che compongono tale copertura.

Ipotesi: Sia $A(G, k)$ un algoritmo "oracolo" per VC_{decision} che accetta in input un grafo G e un intero k , e restituisce:

- **YES:** se esiste un Vertex Cover di dimensione $\leq k$.
- **NO:** se non esiste.

Procedimento

L'algoritmo di ricerca si svolge in due fasi sequenziali.

Fase 1: Calcolo della Cardinalità Minima (k_{opt})

L'obiettivo è determinare la dimensione minima esatta del Vertex Cover.

Poiché la dimensione della soluzione è un intero compreso tra 0 e $|V|$, utilizziamo una Ricerca Binaria interrogando l'oracolo A .

1. Inizializziamo l'intervallo di ricerca: $min = 0, max = |V|$.
2. Mentre $min < max$:
 - Poniamo $mid = \lfloor \frac{min+max}{2} \rfloor$.
 - Eseguiamo $A(G, mid)$.
 - Se l'esito è **YES**: la soluzione ottima è $\leq mid$, quindi poniamo $max = mid$.
 - Se l'esito è **NO**: la soluzione ottima è $> mid$, quindi poniamo $min = mid + 1$.
3. Al termine, $k_{opt} = min$.

Costo Fase 1: $O(\log |V|) \times Costo(A)$.

Fase 2: Costruzione dell'Insieme di Copertura (C)

L'obiettivo è individuare quali specifici vertici compongono l'insieme C di cardinalità k_{opt} .

Iteriamo su ogni vertice $v \in V$ del grafo originale $G = (V, E)$.

1. Inizializziamo $C = \emptyset$ e $k = k_{opt}$.
2. Per ogni vertice $v \in V$:
 - Costruiamo temporaneamente il grafo $G' = G \setminus \{v\}$ (rimuovendo v e i suoi archi incidenti).
 - Interroghiamo l'oracolo: $A(G', k - 1)$.
 - **Analisi della risposta:**

- **Esito YES:** Significa che è possibile coprire i restanti archi con i $k - 1$ vertici rimanenti. Dunque, v può far parte di una soluzione ottima.
 - *Azione:* Aggiungiamo v a C ($C \leftarrow C \cup \{v\}$).
 - Decrementiamo il budget ($k \leftarrow k - 1$).
 - Aggiorniamo il grafo corrente rimuovendo definitivamente v ($G \leftarrow G'$).
- **Esito NO:** Significa che rimuovendo v , il resto del grafo richiede ancora k vertici per essere coperto. Non abbiamo "guadagnato" nulla spendendo un vertice su v .
 - *Azione:* v non viene incluso in C . Il grafo G e il budget k rimangono invariati per la prossima iterazione.

3. Restituiamo C .

Costo Fase 2: $O(|V|) \times Costo(A)$.

Conclusione sulla Complessità

L'intero procedimento richiede un numero di chiamate all'oracolo pari a $O(\log |V| + |V|)$, che è polinomiale rispetto alla dimensione dell'input. Pertanto, la riduzione è polinomiale.

Approssimazione di Vertex Cover

Ci sono vari modi in cui possiamo andare a definire un algoritmo di approssimazione:

- Algoritmi greedy
- Programmazione Lineare intera

Algoritmi greedy

La strategia **Greedy** (ingorda) è spesso la prima tecnica utilizzata per tentare di approssimare problemi di ottimizzazione: si costruisce la soluzione passo dopo passo, effettuando ad ogni iterazione la scelta che appare localmente migliore. Tuttavia, per il problema del Vertex Cover, la definizione di "scelta migliore" è determinante. Analizziamo due euristiche naturali e dimostriamo perché **non garantiscono** un fattore di approssimazione costante.

La strategia consiste nel *scegliere un determinato nodo* del grafo con una **certa strategia**, poi rimuovere i vari archi collegati. Questo viene ripetuto fino a quando vengono rimossi tutti gli archi del grafo.

Approccio naive

Scelgo un nodo qualsiasi con un grado maggiore di 0


```

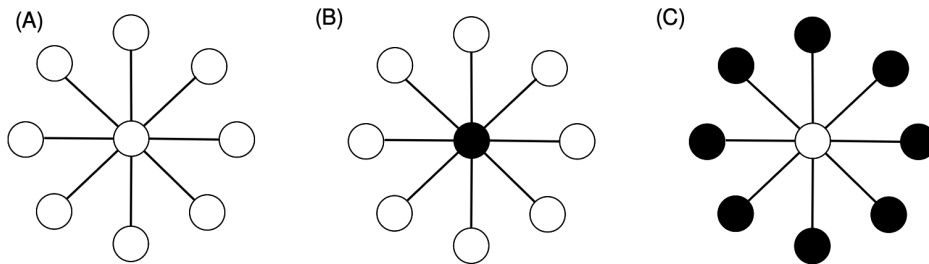
Algorithm Greedy-VC-Naive( $G = (V, E)$ )
   $C = \{\}$ 
  WHILE ( $E$  is not empty) DO
    1. Scegli un vertice arbitrario  $v$  in  $V$  che abbia grado  $> 0$ 
    2.  $C = C \cup \{v\}$ 
    3. Rimuovi da  $E$  tutti gli archi incidenti a  $v$ 
  RETURN  $C$ 

```

Il costo computazionale dell'algoritmo è $O(|V| + |E|)$.

Sebbene l'algoritmo produca sempre una copertura valida in tempo polinomiale, il fattore di approssimazione non è limitato (non è costante).

Controesempio (Grafo a Stella):



Consideriamo un grafo a stella S_k composto da un nodo centrale c collegato a k foglie $l_1, l_2, \dots, l_k$.

- **Soluzione Ottima (C^*):** Basta selezionare il solo nodo centrale c . Dimensione $|C^*| = 1$.
- **Caso Peggior dell'Algoritmo:** L'algoritmo potrebbe sfortunatamente selezionare, uno dopo l'altro, tutti i nodi foglia $l_1, \dots, l_k$. Ogni volta che seleziona una foglia, copre solo un arco.
- **Risultato:** L'algoritmo restituisce un insieme di dimensione k .

Rapporto di Approssimazione:

$$\rho = \frac{|C_{\text{greedy}}|}{|C^*|} = \frac{k}{1} = k$$

Poiché k dipende dal numero di vertici del grafo (n), il rapporto di approssimazione è $O(n)$, il che è **inaccettabile** per un algoritmo di approssimazione.

Approccio euristico

Per ovviare al problema della stella, è naturale raffinare la scelta greedy: invece di un nodo a caso, scegliamo il nodo che "paga di più", ovvero quello che copre il maggior numero di archi in un colpo solo.

```

Algorithm Greedy-VC-MaxDegree( $G = (V, E)$ )
   $C = \{\}$ 
  WHILE ( $E$  is not empty) DO
    1. Scegli il vertice  $v$  in  $V$  con il grado corrente massimo
    2.  $C = C \cup \{v\}$ 
    3. Rimuovi da  $E$  tutti gli archi incidenti a  $v$ 
    4. Aggiorna i gradi dei vertici rimanenti
  RETURN  $C$ 

```

Questa strategia risolve brillantemente il caso della stella (selezionerebbe subito il centro). Tuttavia, è stato dimostrato che anche questa euristica **non ottiene un fattore di approssimazione costante**.

Il Limite Teorico (Perché fallisce):

Mentre per la stella funziona, esistono grafi appositamente costruiti in cui l'algoritmo viene "ingannato" nel preferire una lunga sequenza di nodi a grado medio-alto, invece di pochi nodi ottimi.

È noto in letteratura (dimostrazione di Chvátal, 1979 per Set Cover, applicabile qui) che questo algoritmo ha un fattore di approssimazione logaritmico:

$$\rho \approx \ln(n)$$

Dove $\ln(n)$ è il logaritmo naturale del numero di vertici. Sebbene $\ln(n)$ cresca molto lentamente, non è un numero costante (come 2 o 1.5). Per grafi molto grandi, l'errore può diventare arbitrariamente grande.

Il "Grafo Ingannevole" (Come l'algoritmo viene fregato) (Super approfondimento)

Il Vertex Cover è un caso particolare del Set Cover (dove gli "elementi da coprire" sono gli archi e gli "insiemi" sono i nodi). Poiché l'approccio greedy per il Set Cover ha un'approssimazione pari a H_d (H_d è l' n -esimo numero armonico, dove d è il grado massimo), il fattore di approssimazione è effettivamente $O(\log n)$.

Per visualizzare il limite teorico, è utile capire come viene costruito il grafo che "inganna" l'algoritmo.

Immagina un grafo bipartito con due insiemi di nodi, **A** e **B**:

1. **L'insieme A** contiene un numero piccolo di nodi, diciamo k . (Questo sarà il Vertex Cover ottimo).
2. **L'insieme B** contiene molti più nodi, suddivisi in "livelli" basati sul loro grado.
3. Gli archi sono collegati in modo tale che, all'inizio, i nodi in **B** abbiano un grado leggermente

superiore ai nodi in **A**.

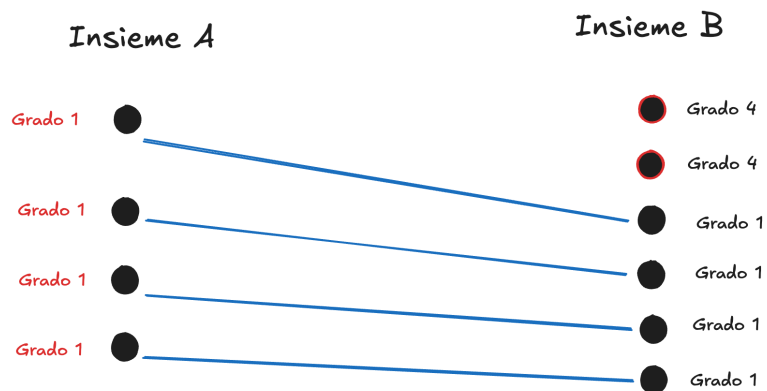
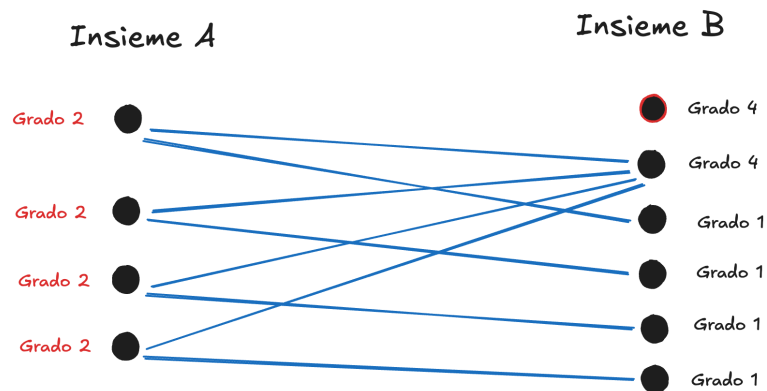
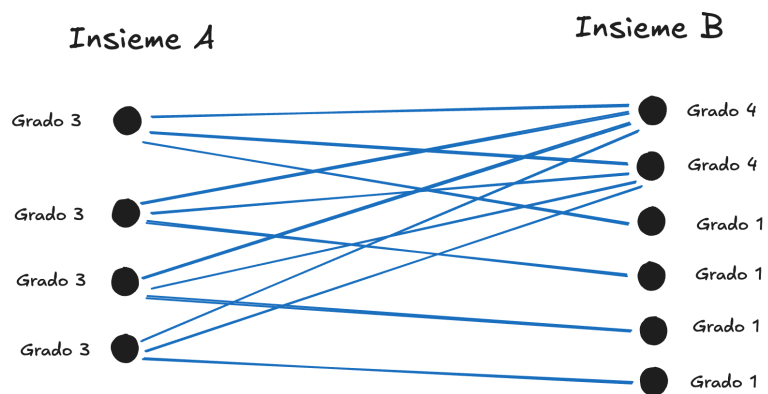
Cosa fa l'algoritmo?

L'algoritmo Greedy guarderà il grafo e dirà: *"I nodi in B hanno grado più alto, ne prendo uno da B"*.

Una volta rimosso quel nodo e i suoi archi, i gradi si aggiornano, ma la struttura è creata ad arte affinché ci sia *sempre* un nodo in **B** che sembra migliore dei nodi in **A**, passo dopo passo.

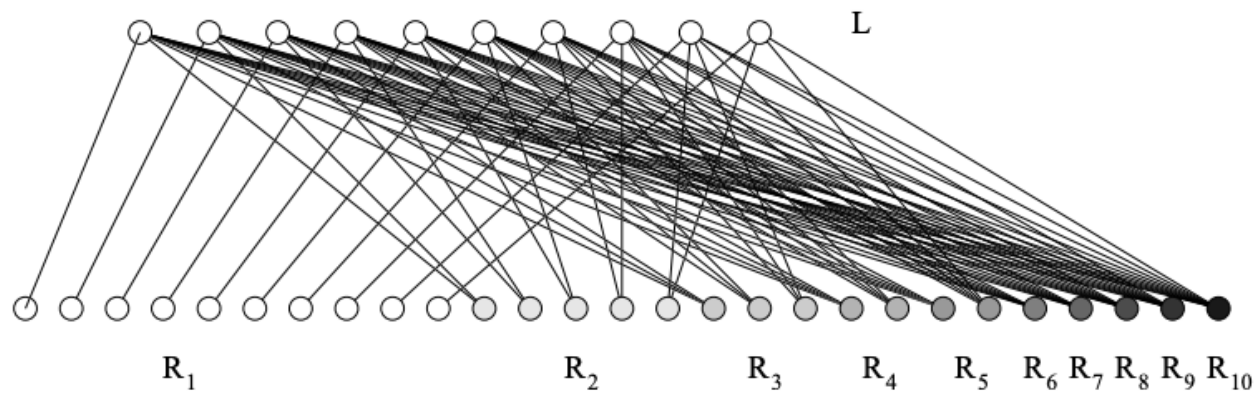
Alla fine, l'algoritmo sceglie quasi tutti i nodi dell'insieme B (che sono tantissimi), mentre la soluzione ottima era prendere semplicemente i pochi nodi dell'insieme A. È così che si accumula il fattore logaritmico $\ln(n)$.

Quello che succede in scala molto piccola è più o meno questo:



Il vero grafo bipartito che inganna l'algoritmo dovrebbe essere questo:

 Caption



Preso da <https://cgi.csc.liv.ac.uk/~michele/TEACHING/COMP309/2005/Lec10.4.4.pdf>

Approccio Euristico corretto: Usiamo gli archi!

L'idea è semplice ma potente: troviamo un insieme di archi disgiunti (che non condividono vertici) e prendiamo entrambi gli estremi di questi archi.

Ecco lo pseudocodice formale:

Plaintext

```

APPROX-VERTEX-COVER(G)
1.  C ← ∅
2.  E' ← E[G]
3.  while E' ≠ ∅ do
4.      Scegli un arco arbitrario (u, v) ∈ E'
5.      C ← C ∪ {u, v}
6.      Rimuovi da E' ogni arco incidente su u o su v
7.  return C

```

Analisi della Complessità:

Il tempo di esecuzione è $O(V + E)$. Possiamo rappresentare il grafo con liste di adiacenza e marcare i vertici man mano che vengono rimossi/coperti. Visitiamo ogni vertice e ogni arco un numero costante di volte.

3. Correttezza e Analisi del Fattore di Approssimazione

Questa è la parte cruciale per l'esame (e dove abbiamo corretto il materiale originale).

Teorema: APPROX-VERTEX-COVER è un algoritmo di approssimazione polinomiale con fattore $\rho = 2$.

Dimostrazione:

Sia C l'insieme dei vertici restituiti dall'algoritmo.

Sia C^* (o OPT) una copertura dei vertici ottima (di cardinalità minima).

Sia A l'insieme degli archi selezionati nella riga 4 dell'algoritmo (questo insieme A costituisce un *Matching Massimale*).

Passo 1: Relazione tra C e A

L'algoritmo inserisce in C esattamente i due estremi di ogni arco scelto in A . Poiché nessun arco viene scelto due volte e gli archi in A sono disgiunti:

$$|C| = 2 \cdot |A| \quad (\text{Eq. 1})$$

Passo 2: Relazione tra C^* e A (Lower Bound)

Qualsiasi Vertex Cover valido deve coprire tutti gli archi di G , inclusi quelli in A .

Poiché gli archi in A sono disgiunti (non condividono vertici), nessun singolo vertice può coprire più di un arco di A .

Di conseguenza, per coprire $|A|$ archi disgiunti, sono necessari almeno $|A|$ vertici distinti.

Quindi, la soluzione ottima deve avere cardinalità:

$$|C^*| \geq |A| \quad (\text{Eq. 2})$$

(Nota: Questo è il passaggio che corregge il refuso delle tue slide).

Passo 3: Conclusione

Combinando l'Eq. 1 e l'Eq. 2:

Dalla (Eq. 2) sappiamo che $|A| \leq |C^*|$. Sostituiamo $|A|$ nella (Eq. 1):

$$|C| = 2 \cdot |A| \leq 2 \cdot |C^*|$$

Riscrivendo in termini di rapporto di approssimazione:

$$\frac{|C|}{|C^*|} \leq 2$$

C.V.D. (Come Volevasi Dimostrare).

4. L'Analisi è "Tight" (Stretta)?

Un professore potrebbe chiederti: *"Possiamo fare meglio di 2 con questo specifico algoritmo?"* o *"L'analisi è pessimistica?"*

La risposta è: **Sì, l'analisi è tight**. Ci sono casi in cui l'algoritmo sbaglia esattamente di un fattore 2.

Esempio Formale (Il Grafo Bipartito Completo $K_{n,n}$):

Immagina un grafo con due insiemi di vertici $U = \{u_1, \dots, u_n\}$ e $W = \{w_1, \dots, w_n\}$, dove tutti gli u sono collegati a tutti i w .

- **Soluzione Ottima (OPT):** Basta prendere tutti i vertici di U (o tutti quelli di W). Cardinalità $= n$.
- **Soluzione Algoritmo Greedy:** L'algoritmo potrebbe sfortunatamente scegliere una serie di archi disgiunti come $(u_1, w_1), (u_2, w_2), \dots, (u_n, w_n)$. Per ogni arco, prende *entrambi* i vertici. Cardinalità $= 2n$.

Rapporto: $2n/n = 2$.

Questo dimostra che non possiamo dimostrare un fattore inferiore a 2 per questo algoritmo specifico.

Il resto è ancora da rielaborare

Il Paradosso del Calcolo (e la sua soluzione)

"Se il problema è NP-Hard e non conosciamo $OPT(I)$, come facciamo a dimostrare che il nostro algoritmo è vicino a $OPT(I)$?"

È come cercare di dimostrare che sei alto quasi quanto una persona invisibile di cui non conosci l'altezza.

La Soluzione: Il Lower Bound (Limite Inferiore)

Per aggirare il problema, usiamo un "proxy" o un'approssimazione dell'ottimo che siamo in grado di calcolare facilmente.

Cerchiamo un valore LB (Lower Bound) che sappiamo per certo essere minore o uguale all'ottimo:

$$LB(I) \leq OPT(I)$$

La strategia di dimostrazione diventa quindi un processo a tre passi:

1. Troviamo un **Lower Bound (LB)** efficiente da calcolare (es. il costo del Minimum Spanning Tree per il TSP).
2. Dimostriamo che il nostro algoritmo A produce una soluzione che è legata a questo LB da un fattore costante (es. $A(I) \leq 2 \cdot LB$).
3. Usiamo la proprietà transitiva:

$$A(I) \leq 2 \cdot LB(I) \leq 2 \cdot OPT(I)$$

Ecco fatto! Abbiamo dimostrato che $\rho = 2$ senza mai conoscere il vero valore di OPT .

$2 \cdot LB$ è un **upper bound (limite superiore)** del costo del tuo algoritmo, ma con una caratteristica speciale: è un upper bound che **possiamo calcolare**.

Proviamo a sviscerare questa intuizione, perché è il cuore dell'analisi.

La catena della sicurezza

Immagina di voler dimostrare che il tuo algoritmo non spende mai più del doppio dell'ottimo ($2 \cdot OPT$). Il problema è che OPT è un fantasma: non sai quanto vale.

Quindi costruisci questa catena logica:

1. **Il costo del tuo algoritmo (ALG):** È *esattamente* (o al massimo) $2 \cdot LB$.
 - Nel Vertex Cover: $ALG = 2 \cdot |E'|$.
 - Questo valore lo conosci! Se il tuo matching ha 50 archi, sai che il tuo algoritmo userà 100 vertici.
2. **Il confronto con il fantasma (OPT):**
 - Sappiamo che $LB \leq OPT$ (perché l'ottimo deve coprire almeno gli archi del matching).
 - Moltiplichiamo tutto per 2: $2 \cdot LB \leq 2 \cdot OPT$.

Mettiamo tutto insieme

$$ALG = \underbrace{2 \cdot LB}_{\text{Limite calcolabile}} \leq \underbrace{2 \cdot OPT}_{\text{Limite ideale}}$$

Quindi sì, $2 \cdot LB$ funge da "tetto" sicuro.

Tu puoi dire: "Non so quanto sia l'ottimo, ma so che il mio algoritmo spende $2 \cdot LB$. E dato che $2 \cdot LB$ è sicuramente più piccolo di $2 \cdot OPT$, sono salvo: non ho sfiorato il fattore 2."

Perché è geniale?

Stai usando una quantità "pessimistica" per l'ottimo.

Dire che $OPT \geq LB$ è come dire "L'ottimo è almeno grande quanto il matching".

Anche nella peggiore delle ipotesi (in cui l'ottimo fosse piccolo, cioè proprio uguale a LB), il tuo algoritmo sarebbe comunque "solo" il doppio ($2 \cdot LB$).

Se l'ottimo fosse più grande di LB , tanto meglio! Il tuo algoritmo sarebbe ancora più vicino all'ottimo di quanto pensi (il fattore reale sarebbe minore di 2).

Esempio Pratico: Calcolo del fattore per TSP Metrico (2-Approx)

Applichiamo subito il metodo al **TSP Metrico** usando l'algoritmo basato sull'MST (Minimum Spanning Tree), che abbiamo citato prima.

Passo 1: Identificare il Lower Bound (LB)

Immagina la soluzione ottima del TSP: è un ciclo che tocca tutti i nodi.

Se rimuoviamo un arco qualsiasi da questo ciclo ottimo, otteniamo un cammino che tocca tutti i nodi (uno Spanning Path).

Uno Spanning Path è un tipo di albero di copertura.

Poiché l'MST è l'albero di copertura di costo minimo assoluto, il suo costo deve essere inferiore o uguale al costo di quel cammino, e quindi inferiore al ciclo ottimo.

$$Costo(MST) \leq Costo(OPT)$$

Abbiamo il nostro $LB = Costo(MST)$.

Passo 2: Analizzare l'algoritmo

L'algoritmo "raddoppia" gli archi dell'MST per creare un grafo Euleriano e poi trova un ciclo.

Il costo del grafo raddoppiato è esattamente $2 \cdot Costo(MST)$.

Grazie alle scorciatoie (che non aumentano il costo per la disuguaglianza triangolare), il tour finale $A(I)$ non costerà più del grafo raddoppiato:

$$A(I) \leq 2 \cdot Costo(MST)$$

Passo 3: Conclusione (Calcolo di ρ)

Mettiamo insieme i pezzi:

$$A(I) \leq 2 \cdot Costo(MST) \leq 2 \cdot Costo(OPT)$$

Dividendo tutto per OPT (che è positivo):

$$\frac{A(I)}{OPT(I)} \leq 2$$

Quindi, il fattore di approssimazione è **2**.

Classi di Approssimabilità (Approfondimento)

Non tutti i problemi sono uguali. In base a che tipo di ρ riusciamo a ottenere, classifichiamo i problemi in classi di complessità (trovi i dettagli nelle slide "Complexity Theory"):

1. **APX (Approximable)**: Problemi che ammettono un'approssimazione con ρ costante (es. Vertex Cover, TSP Metrico).
 2. **PTAS (Polynomial Time Approximation Scheme)**: Possiamo scegliere quanto essere precisi. Per ogni $\epsilon > 0$, esiste un algoritmo con fattore $(1 + \epsilon)$. Il tempo però può dipendere esponenzialmente da $1/\epsilon$.
 3. **FPTAS (Fully PTAS)**: Il Santo Graal. Come sopra, ma il tempo è polinomiale anche rispetto a $1/\epsilon$ (es. Knapsack Problem).
 4. **Non-Approssimabili (Log-APX o peggio)**: Problemi dove ρ dipende dalla dimensione dell'input n . Esempio: **Set Cover** ha un fattore $\ln(n)$.
-

Algoritmo 2-Approx (Approfondimento)

Questo è l'approccio più intuitivo. L'idea è: *"Non so trovare il ciclo perfetto, ma so trovare l'albero più economico che collega tutti. Partiamo da lì."*

I Passaggi (Step-by-Step):

1. **Minimum Spanning Tree (MST)**: Calcola l'Albero di Copertura Minimo del grafo G . Sia T questo albero.
 - *Perché?* Perché $Cost(T) \leq Cost(OPT)$. È il modo più economico per connettere tutto, anche se non è un ciclo.
2. **Raddoppiamento (Doubling)**: Raddoppia ogni arco di T . Ottieni un multigrafo G' dove ogni arco dell'albero appare due volte.
 - *Perché?* In un grafo, un **Ciclo Euleriano** (un giro che attraversa ogni arco esattamente una volta e torna all'inizio) esiste se e solo se tutti i nodi hanno **grado pari**. Raddoppiando gli archi, ogni nodo avrà grado $2 \times k$, quindi sicuramente pari!
3. **Ciclo Euleriano**: Trova un ciclo euleriano E in G' . Questo è facile da fare in tempo lineare.

4. **Scorciatoie (Shortcutting):** Trasforma il ciclo euleriano in un Ciclo Hamiltoniano. Scorri la lista dei nodi del ciclo euleriano; se incontri un nodo che hai già visitato, saltalo e vai direttamente al successivo non visitato.
- *Il trucco:* Grazie alla **disuguaglianza triangolare**, saltare un nodo (andare da u a w invece di fare $u \rightarrow v \rightarrow w$) non aumenta mai il costo.

Analisi del Costo (Il fattore 2):

- Costo MST $\leq 1 \cdot OPT$.
 - Grafo Raddoppiato = $2 \cdot \text{Costo}(MST) \leq 2 \cdot OPT$.
 - Le scorciatoie non peggiorano il costo.
 - **Risultato:** La soluzione è $\leq 2 \cdot OPT$.
-

2. Algoritmo di Christofides (1.5-Approx)

Nicos Christofides (nel 1976) si chiese: *"Raddoppiare tutti gli archi è uno spreco enorme. Possiamo rendere i gradi pari aggiungendo meno peso?"*

La risposta è sì, lavorando solo sui nodi "problematici".

I Passaggi (Step-by-Step):

1. **Minimum Spanning Tree (MST):** Calcola l'MST, T . (Come prima).
2. **Individua i Nodi Dispari:** Chiama O l'insieme dei nodi che in T hanno grado dispari.
 - *Teorema:* Per il "Lemma delle strette di mano", il numero di nodi con grado dispari in un grafo è sempre **pari**. Quindi $|O|$ è un numero pari.
3. **Minimum Weight Perfect Matching:** Costruisci un **Matching Perfetto di Costo Minimo** solo sui nodi in O . Chiamiamo questo insieme di archi M .
 - In pratica: accoppiamo i nodi dispari tra loro spendendo il meno possibile.
4. **Unione:** Aggiungi gli archi del matching M all'albero T . Ottieni un multigrafo $H = T \cup M$.
 - *Risultato:* I nodi che avevano grado pari in T non sono stati toccati (o toccati due volte se facevano parte del matching, restando pari). I nodi che avevano grado dispari in T hanno ricevuto esattamente un arco in più dal matching, diventando pari. **Tutti i nodi ora hanno grado pari!**
5. **Ciclo Euleriano & Shortcut:** Come nell'algoritmo precedente, trova il ciclo euleriano in H e applica le scorciatoie.

Analisi del Costo (Il fattore 1.5):

Qui serve attenzione, è la parte che distingue il 30 dalla lode.

- Sappiamo che $Costo(T) \leq 1 \cdot OPT$.
- **Quanto costa il Matching M ?**
 - Immagina il ciclo ottimo OPT solo sui nodi dispari O . Questo ciclo è formato da due matching alternati.
 - Il matching che abbiamo trovato noi è quello di costo minimo, quindi non può costare più della metà del ciclo ottimo ristretto a quei nodi.
 - Poiché (per disuguaglianza triangolare) il ciclo sui nodi O non costa più del ciclo su *tutti* i nodi, abbiamo:
 - $Costo(M) \leq \frac{1}{2} Costo(OPT)$.
- Totale:

$$Costo(Christofides) = Costo(T) + Costo(M) \leq 1 \cdot OPT + 0.5 \cdot OPT = 1.5 \cdot OPT$$
